DJANGO INTRODUCTION

Contents

- What is Django?
- History
- Pronunciation
- Django Stablility?
- Which sites use Django?
- Installation

Contents..

➢Create new Application

➢Steps to create a webpage ➢Create view.py form ➢Create URLS ➢Web Page

What is Django?

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design.

What is Django?

Python Web framework

Quick Development

Stability

Less code

Don't repeat yourself (DRY)

Migration

Automate the repetitive stuff

Object-Relational Mapper

Model ,View , Template

Scalable

Catch Framework

Loose coupling

Follow best practices

Explicit is better than implicit

What is Django?

The main reason behind Django's existence is that Django inherited Python's "batteries-included" approach.

It also includes pre-made modules and applications for common tasks in web development like

- User authentication,
- templates,
- routes, and views,
- admin interface,
- robust security and
- support for multiple database backends.

Why Learn Django?

Django offers lots of features and is a new emerging technology of the future.

Since it is based on **python**, which itself is a very powerful language and also is going to be used in the future extensively, therefore, it is worthwhile to learn Django.

Why Learn Django?

Django has Evolved Over Time

Since the release of Django, it is having a lot of features and still, Django continues its journey and has been in the industry for **more than a decade.**

https://docs.djangoproject.com/en/2.2/releases/

Is Django stable?

Yes, it's quite stable. Companies like **Disqus, Instagram, Pinterest, and Mozilla** have been using Django for many years. Sites built on Django have weathered traffic spikes of over 50 thousand hits per second.

Why Learn Django?

Ridiculously fast.

Django was designed to help developers take applications from concept to completion as quickly as possible.

Reassuringly secure.

Django takes security seriously and helps developers avoid many common security mistakes.

Exceedingly scalable.

Some of the busiest sites on the Web leverage Django's ability to quickly and flexibly scale.

Why Learn Django?

Fully loaded.

Django includes dozens of extras you can use to handle common Web development tasks. Django takes care of user authentication, content administration, site maps, RSS feeds, and many more tasks — right out of the box.

Reassuringly secure.

Django takes security seriously and helps developers avoid many common security mistakes, such as SQL injection, cross-site scripting, cross-site request forgery and clickjacking. Its user authentication system provides a secure way to manage user accounts and passwords.

Why Learn Django?

Exceedingly scalable.

Some of the busiest sites on the planet use Django's ability to quickly and flexibly scale to meet the heaviest traffic demands.

Incredibly versatile.

Companies, organisations and governments have used Django to build all sorts of things — from content management systems to social networks to scientific computing platforms.

Why Learn Django?

https://www.djangosites.org/

https://www.djangosites.org/with-source/

https://www.djangosites.org/stats/

MTV Framework

Model - Data access layer.

This deals with access, validation and relationships among data. Models are Python classes that mediate between Django ORM and the database tables.

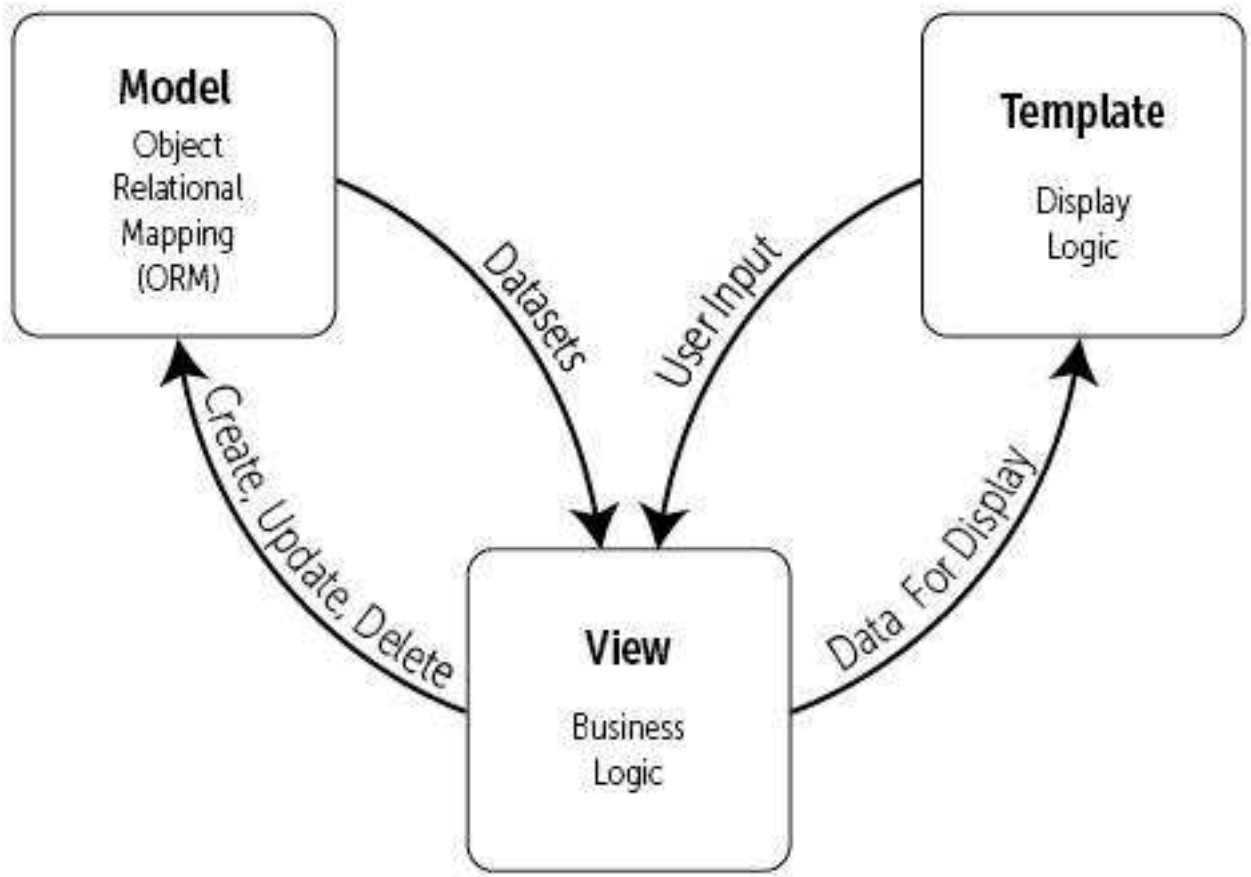
Template - Presentation layer.

This deals with how data is displayed to the client.

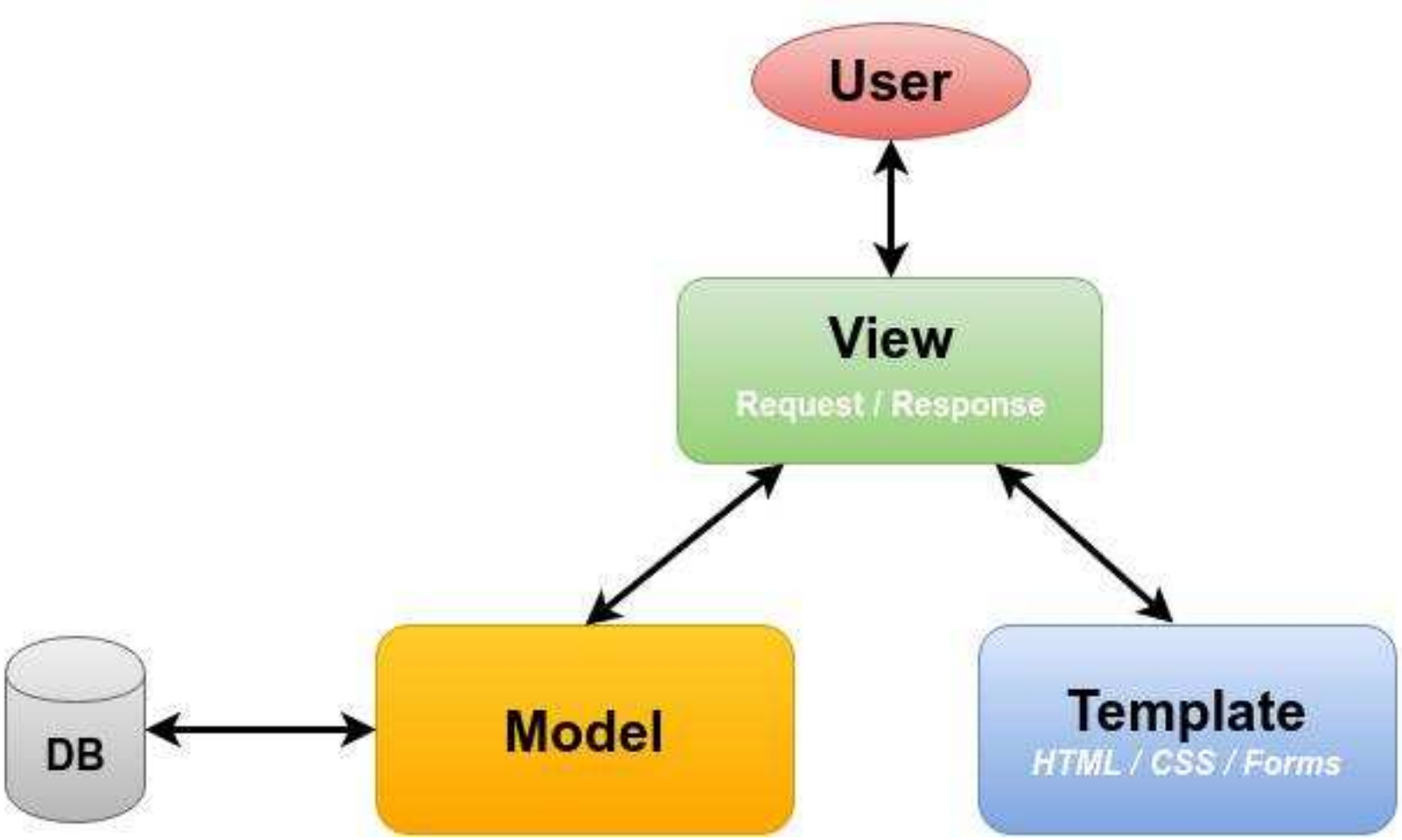
View - Business logic layer.

This is a bridge between models and templates. A view accesses model data and redirects it to a template for presentation. Unlike the definition of a view in traditional MVC architecture, a Django view describes which data you see, not how you see it. It's about processing, not presentation.

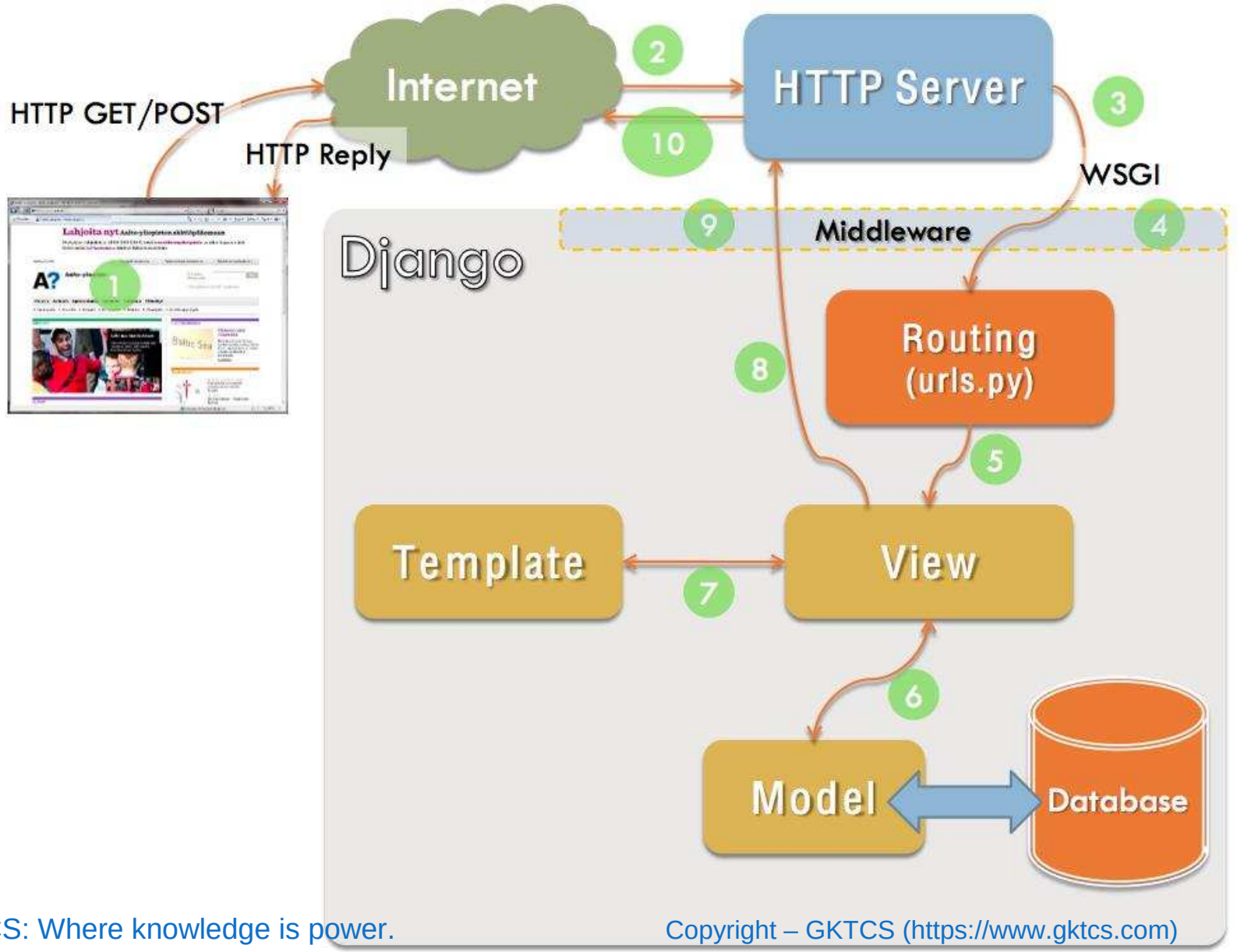
MTV Framework



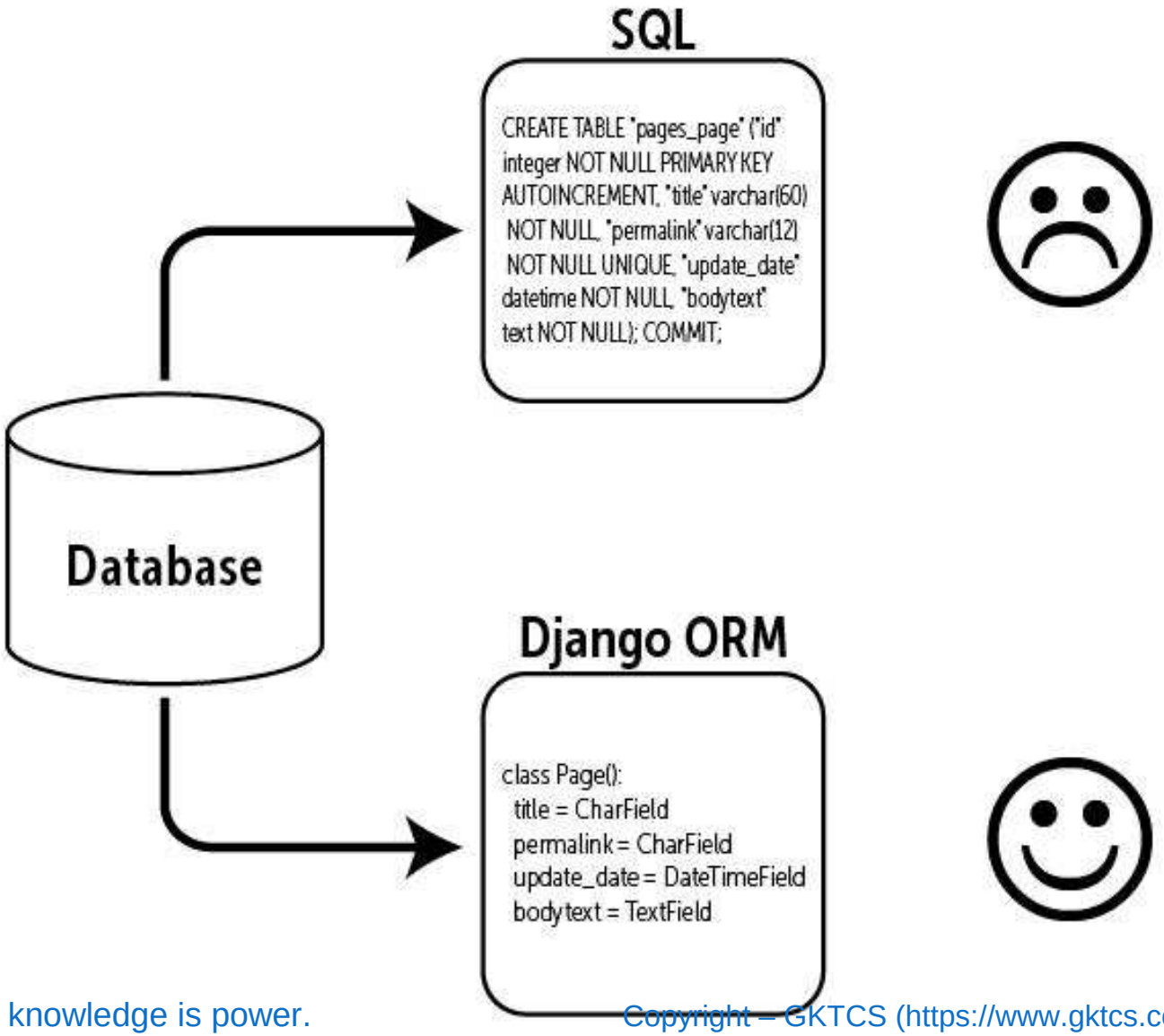
MTV Framework



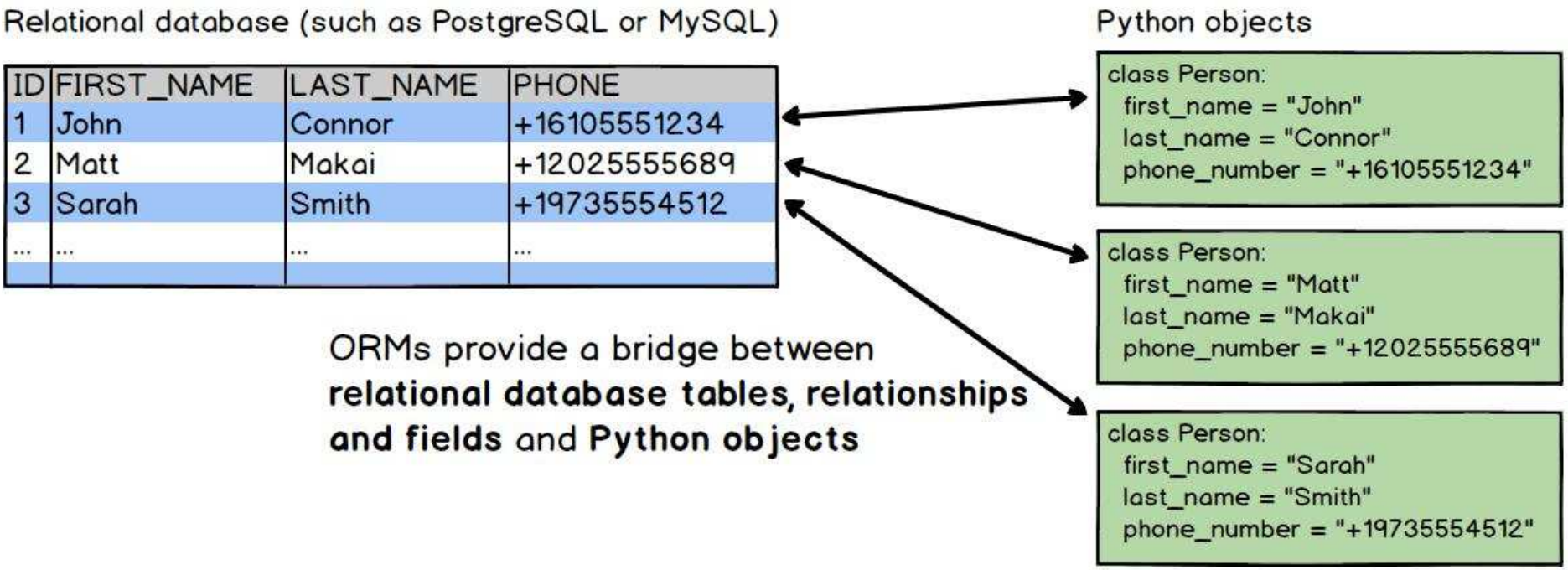
MTV Framework



Django ORM



Django ORM



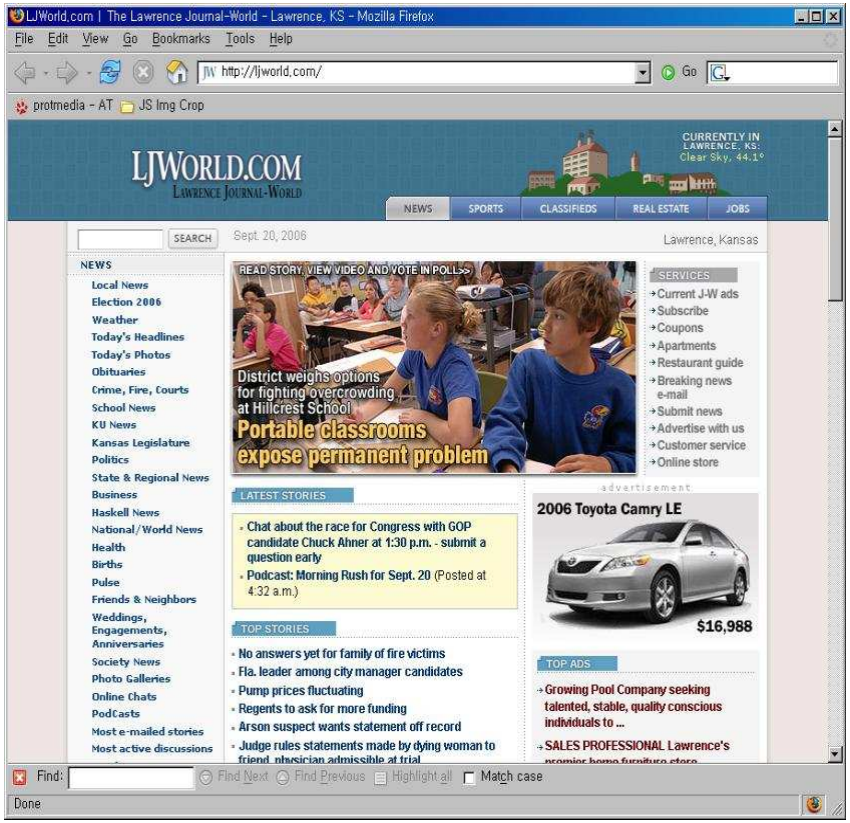
Django ORM

web framework	None	Flask	Flask	Django
ORM	SQLAlchemy	SQLAlchemy	SQLAlchemy	Django ORM
database connector	(built into Python stdlib)	MySQL-python	psycopg	psycopg
relational database	 SQLite	 MySQL	 PostgreSQL	 PostgreSQL

History

Lawrence Journal-World by World Online Developers.

- ✓ **http://www.ljworld.com**
- ✓ **http://www.Lawrence.com**
- ✓ **http://www.KUsports.com**



Pronunciation

Django Reinhardt

Gypsy jazz guitarist (From the 1930s to early 1950s)

Django is pronounced JANG-oh.
Rhymes with FANG-oh. The “D” is silent.

Installation and Setup

Django Pre-Requisites

- Python 2.x / Python 3.x
- Database drivers for the database you wish to use ➢PostgreSQL, Oracle, SQLite, MySQL
- Web Server
  ➢Django has a built-in web server for development

pip install django

https://pypi.org/project/mysqlclient/

Installation

```
# settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'OPTIONS': {
            'read_default_file': '/path/to/my.cnf',
        },
    }
}


# my.cnf
[client]
database = NAME
user = USER
password = PASSWORD
default-character-set = utf8
```

Installation

```
DATABASES = {
    'default': {
        'ENGINE':
'django.db.backends.oracle',
        'NAME': 'xe',
        'USER': 'a_user',
        'PASSWORD': 'a_password',
        'HOST': '',
        'PORT': '',
    }
}
```

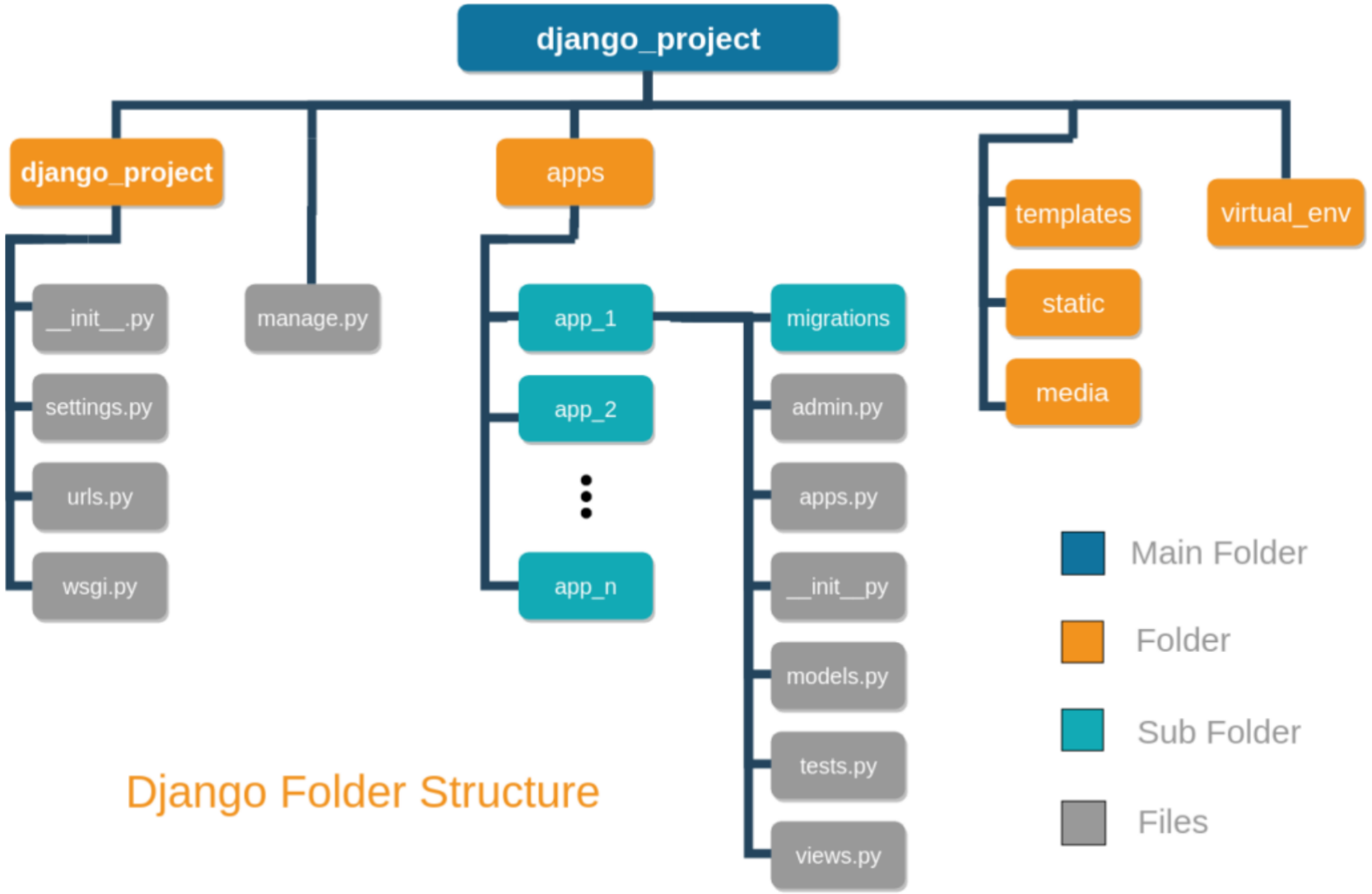
Getting Started with Django

```
$ python -m django –version

$ django-admin startproject mysite

$ python manage.py runserver

$ python manage.py runserver 8080

$ python manage.py runserver 0:8000
```

Getting Started with Django

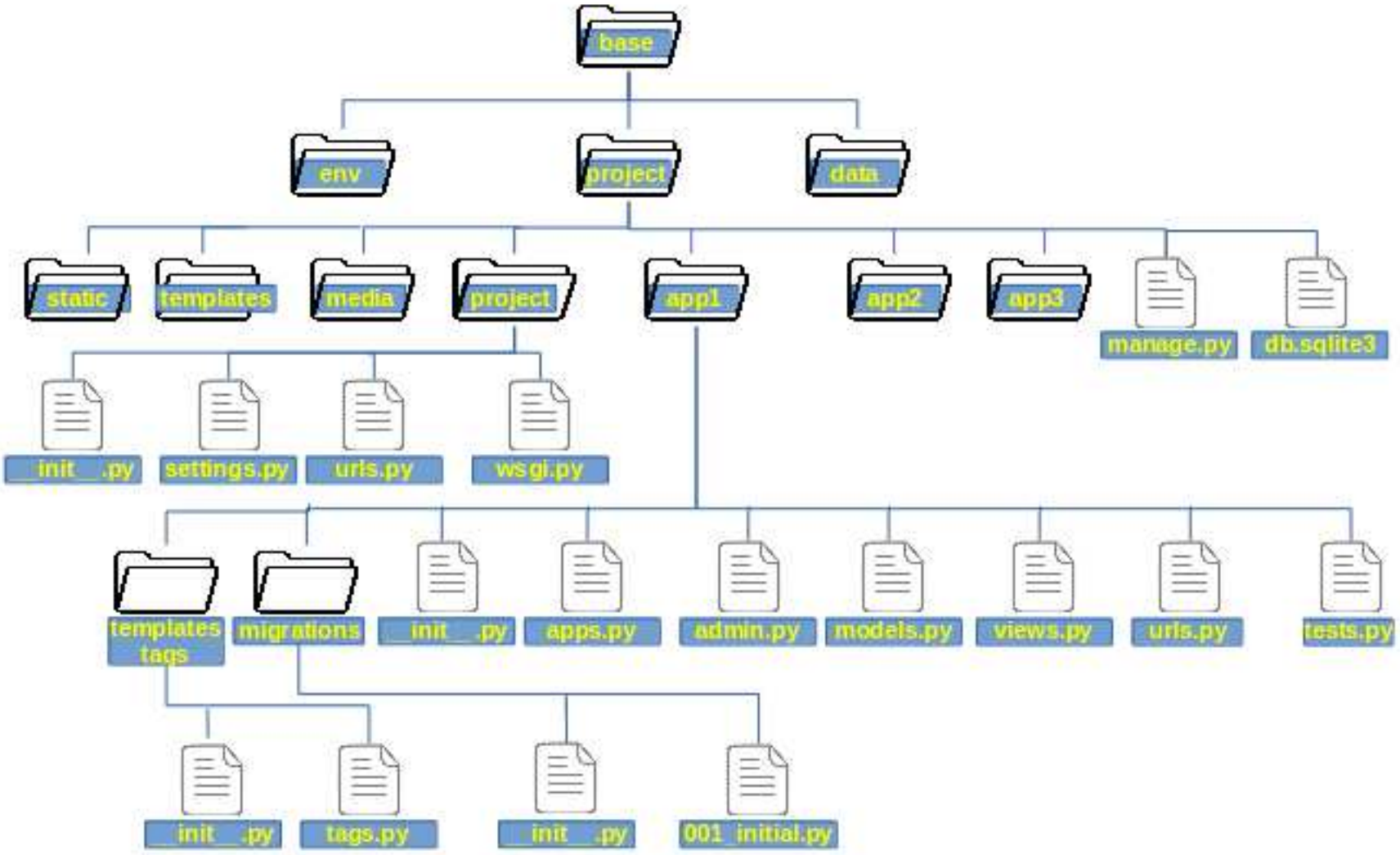
```
$ python manage.py startapp polls

polls/
    __init__.py
    admin.py
    apps.py
    migrations/
        __init__.py
    models.py
    tests.py
    views.py
```

Django Project Layout



Django Folder Structure

Django Project Layout



```
mysite/
    manage.py
    mysite/
        __init__.py
        settings.py
        urls.py
        wsgi.py
    polls/
        __init__.py
        admin.py
        migrations/
            __init__.py
            0001_initial.py
        models.py
        static/
            polls/
                images/
                    background.gif
                style.css
        templates/
            polls/
                detail.html
                index.html
                results.html
        tests.py
        urls.py
        views.py
    templates/
        admin/
            base_site.html
```

Getting Started with Django

polls/views.py

```
from django.http import HttpResponse


def index(request):
    return HttpResponse("Hello, world.
 You're at the polls index.")
```

Getting Started with Django

polls/urls.py

```
from django.urls import path

from . import views

urlpatterns = [
    path('', views.index, name='index'),
]
```

Getting Started with Django

mysite/urls.py

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('polls/', include('polls.urls')),
    path('admin/', admin.site.urls),
]
```

Getting Started with Django

The **include()** function allows referencing other URLconfs.

Whenever Django encounters **include()**, it chops off whatever part of the URL matched up to that point and sends the remaining string to the included URLconf for further processing.

The idea behind **include()** is to make it easy to plug-and-play URLs.
Since polls are in their own URLconf (**polls/urls.py**), they can be placed under “/polls/”.

Getting Started with Django

The **`path()`** function is passed four arguments, two required: **`route`** and **`view`**, and two optional: **`kwargs`**, and **`name`**. At this point, it's worth reviewing what these arguments are for.

`path()` argument: `route`

`route` is a string that contains a URL pattern. When processing a request, Django starts at the first pattern in **`urlpatterns`** and makes its way down the list, comparing the requested URL against each pattern until it finds one that matches.

Getting Started with Django

Patterns don’t search GET and POST parameters, or the domain name.

For example, in a request to

`https://127.0.0.1:8000/myapp/`, the URLconf will look for **`myapp/`**.

In a request to **`https://127.0.0.1:8000/myapp/?page=3`**, the URLconf will also look for **`myapp/`**.

Getting Started with Django

`path()` argument: `view`

When Django finds a matching pattern, it calls the specified view function with an **`HttpRequest`** object as the first argument and any “captured” values from the route as keyword arguments.

`path()` argument: `kwargs`

Arbitrary keyword arguments can be passed in a dictionary to the target view.

`path()` argument: `name`

Naming your URL lets you refer to it unambiguously from elsewhere in Django, especially from within templates.

Getting Started with Django

Database setup

`mysite/settings.py`

If you wish to use another database, install the appropriate database bindings and change the following keys in the **`DATABASES`** 'default' item to match your database connection settings:

- **`ENGINE`** –
  Either **`'django.db.backends.sqlite3'`**, **`'django.db.backends.postgresql'`**, **`'django.db.backends.mysql'`**,
  or **`'django.db.backends.oracle'`**. Other backends are also available.

- **`NAME`** – The name of your database. If you're using SQLite, the database will be a file on your computer; in that case, **`NAME`** should be the full absolute path, including filename, of that file. The default value, **`os.path.join(BASE_DIR, 'db.sqlite3')`**, will store the file in your project directory.

Getting Started with Django

By default, **`INSTALLED_APPS`** contains the following apps, all of which come with Django:

- **`django.contrib.admin`** – The admin site. You'll use it shortly.
- **`django.contrib.auth`** – An authentication system.
- **`django.contrib.contenttypes`** – A framework for content types.
- **`django.contrib.sessions`** – A session framework.
- **`django.contrib.messages`** – A messaging framework.
- **`django.contrib.staticfiles`** – A framework for managing static files.

Getting Started with Django

```
$ python manage.py migrate
```

The **`migrate`** command looks at the **`INSTALLED_APPS`** setting and creates any necessary database tables according to the database settings in your **`mysite/settings.py`** file and the database migrations shipped with the app .

You'll see a message for each migration it applies. If you're interested, run the command-line client for your database and type **`\dt`** (PostgreSQL),
`SHOW TABLES;` (MySQL),
`.schema` (SQLite),
or **`SELECT TABLE_NAME FROM USER_TABLES;`** (Oracle) to display the tables Django created.

Getting Started with Django

```
#polls/models.py
from django.db import models

class Question(models.Model):
    question_text = models.CharField(max_length=200)
    pub_date = models.DateTimeField('date published')

class Choice(models.Model):
    question = models.ForeignKey(Question,
on_delete=models.CASCADE)
    choice_text = models.CharField(max_length=200)
    votes = models.IntegerField(default=0)
```

Getting Started with Django

mysite/settings.py

```
INSTALLED_APPS = [
    'polls.apps.PollsConfig',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

To include the app in our project, we need to add a reference to its configuration class in the **INSTALLED_APPS** setting. The **PollsConfig** class is in the **polls/apps.py** file, so its dotted path is **'polls.apps.PollsConfig'**. Edit the **mysite/settings.py** file and add that dotted path to the **INSTALLED_APPS** setting.

Getting Started with Django

```
$ python manage.py makemigrations polls

Migrations for 'polls':
  polls/migrations/0001_initial.py:
    - Create model Choice
    - Create model Question
    - Add field question to choice


$ python manage.py sqlmigrate polls 0001
```

Getting Started with Django

```
$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes,
polls, sessions
Running migrations:
  Rendering model states... DONE
  Applying polls.0001_initial... OK
```

- Change your models (in **`models.py`**).
- Run **`python manage.py makemigrations`** to create migrations for those changes
- Run **`python manage.py migrate`** to apply those changes to the database.

Getting Started with Django

```
$ python manage.py shell

>>> from polls.models import Choice, Question

>>> Question.objects.all()
<QuerySet []>

>>> from django.utils import timezone
>>> q = Question(question_text="What's
new?", pub_date=timezone.now())

# Save the object into the database. You have to
call save() explicitly.
>>> q.save()

# Now it has an ID.
>>> q.id
1
```

Getting Started with Django

```
# Access model field values via Python attributes.
>>> q.question_text
"What's new?"
>>> q.pub_date
datetime.datetime(2019, 2, 26, 13, 0, 0,
775217, tzinfo=<UTC>)

# Change values by changing the attributes,
then calling save().
>>> q.question_text = "What's up?"
>>> q.save()

# objects.all() displays all the questions in
the database.
>>> Question.objects.all()
<QuerySet [<Question: Question object (1)>]>
```

Getting Started with Django

Wait a minute. **<Question: Question object (1)>** isn't a helpful representation of this object. Let's fix that by editing the **Question** model (in the **polls/models.py** file) and adding a **__str__()** method to both **Question** and **Choice**:

polls/models.py¶

```
from django.db import models

class Question(models.Model):
    # ...
    def __str__(self):
        return self.question_text

class Choice(models.Model):
    # ...
    def __str__(self):
        return self.choice_text
```

Getting Started with Django

Let's also add a custom method to this model:

polls/models.py¶

```
import datetime

from django.db import models from
django.utils import timezone


class Question(models.Model):
    # ...
    def was_published_recently(self):
        return self.pub_date >= timezone.now()
- datetime.timedelta(days=1)
```

Getting Started with Django

Note the addition of **`import datetime`** and

`from django.utils import timezone` , to reference Python's standard **`datetime`** module and Django's time-zone-related utilities in **`django.utils.timezone`**, respectively.

Getting Started with Django

$python manage.py shell

```
>>> from polls.models import Choice, Question

# Make sure our __str__() addition worked.

>>> Question.objects.all()
<QuerySet [<Question: What's up?>]>

# Django provides a rich database lookup API that's
entirely driven by
# keyword arguments.
>>> Question.objects.filter(id=1)
<QuerySet [<Question: What's up?>]>
>>>
Question.objects.filter(question_text__startswith='What')
<QuerySet [<Question: What's up?>]>
```

Getting Started with Django

```
# Get the question that was published this year.
>>> from django.utils import timezone
>>> current_year = timezone.now().year
>>> Question.objects.get(pub_date__year=current_year)
<Question: What's up?>

# Request an ID that doesn't exist, this will raise an
exception.
>>> Question.objects.get(id=2)
Traceback (most recent call last):
    ...
DoesNotExist: Question matching query does not exist.

# Lookup by a primary key is the most common case, so
Django provides a
# shortcut for primary-key exact lookups.
# The following is identical to
Question.objects.get(id=1).
```

Getting Started with Django

```
>>> Question.objects.get(pk=1)
<Question: What's up?>

# Make sure our custom method worked.
>>> q = Question.objects.get(pk=1)
>>> q.was_published_recently()
True
```

Getting Started with Django

```
>>> q = Question.objects.get(pk=1)

# Display any choices from the related object set --
none so far.
>>> q.choice_set.all()
<QuerySet []>

# Create three choices.
>>> q.choice_set.create(choice_text='Not much', votes=0)
<Choice: Not much>
>>> q.choice_set.create(choice_text='The sky', votes=0)
<Choice: The sky>
>>> c = q.choice_set.create(choice_text='Just hacking
again', votes=0)

# Choice objects have API access to their related
Question objects.
```

Getting Started with Django

```
>>> c.question
<Question: What's up?>

# And vice versa: Question objects get
access to Choice objects.
>>> q.choice_set.all()
<QuerySet [<Choice: Not much>, <Choice: The
sky>, <Choice: Just hacking again>]>
>>> q.choice_set.count()
3
```

Getting Started with Django

```
>>Choice.objects.filter(question__pub_date__
year=current_year)
<QuerySet [<Choice: Not much>, <Choice: The
sky>, <Choice: Just hacking again>]>

# Let's delete one of the choices. Use
delete() for that.
>>> c =
q.choice_set.filter(choice_text__startswith=
'Just hacking')
>>> c.delete()
```

Queries ??